

CICD_Standard

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

- 1. Overview
 - 1.1 Purpose
 - 1.2 Scope
- 2. Branching Strategy
 - 2.1 Recommended: Trunk-Based Development
 - 2.2 Branch Naming
 - 2.3 Branch Protection
- 3. Build Pipeline
 - 3.1 Pipeline Stages
 - 3.2 Stage Requirements
 - 3.3 CI Script Example
- 4. Artifact Management
 - 4.1 Artifact Types
 - 4.2 Naming Convention
 - 4.3 Versioning
- 5. Environment Promotion
 - 5.1 Environment Definitions
 - 5.2 Promotion Flow
 - 5.3 Approval Matrix
- 6. Quality Gates
 - 6.1 Gate Rules
 - 6.2 Gate Exemptions
- 7. Appendix
 - 7.1 Reference Documents
 - 7.2 Recommended Tools

CICD Standard

> Reference: Google Continuous Delivery Practices, ThoughtWorks Technology Radar, Alibaba Cloud CICD Best Practices

1. Overview

1.1 Purpose

Standardize the continuous integration and continuous delivery process, ensuring automation, traceability, and high quality throughout the code-to-production lifecycle.

1.2 Scope

Applies to all internally developed systems' code repository management, build pipelines, artifact management, and environment promotion processes.

2. Branching Strategy

2.1 Recommended: Trunk-Based Development

For most insurance core system teams (5-20 people), Trunk-Based Development is recommended:

```
main (trunk)
├─ feature/xxx (short-lived, < 3 days)
│   └─ > merge to main
├─ fix/xxx (fix branch)
│   └─ > merge to main
├─ release/v{major}.{minor} (release branch)
└─ > merge to main + create tag
```

2.2 Branch Naming

Branch Type	Naming Convention	Example
Main	main or master	main
Feature	feature/{JIRA-ID}-{summary}	feature/PROJ-123-add-channel
Fix	fix/{JIRA-ID}-{summary}	fix/PROJ-456-fix-npe
Release	release/v{major}.{minor}	release/v2.3
Hotfix	hotfix/{JIRA-ID}-{summary}	hotfix/PROJ-789-critical

2.3 Branch Protection

- Direct push to main is prohibited (requires PR + CI pass + Code Review)
- PR requires at least 1 approval before merging
- All CI pipelines must pass before merge
- Recommended merge method: Squash Merge

3. Build Pipeline

3.1 Pipeline Stages

```
stages:
- lint
- build
- unit-test
```

- code-scan
- package
- integration
- artifact
- deploy-staging
- e2e-test

3.2 Stage Requirements

Stage	Trigger	Timeout	Quality Gate
lint	Push/PR	5 min	No errors
build	Push/PR	10 min	Build success
unit-test	Push/PR	10 min	Coverage > 80%, all pass
code-scan	Push/PR	15 min	No P0/P1 issues
package	Push to main / Tag	10 min	Build success
integration	Push to main	20 min	All pass
e2e-test	Pre-release	30 min	Core scenarios pass
deploy-staging	Tag / manual	10 min	Health check pass

3.3 CI Script Example

```

name: CI Pipeline
on:
  push:
  branches: [main, release/**]
  pull_request:
  branches: [main]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run linter
        run: ./gradlew checkstyleMain
  build:
    runs-on: ubuntu-latest
    needs: [lint]
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: ./gradlew build -x test
      - name: Save artifact
        uses: actions/upload-artifact@v3
    with:
      name: app-jar
      path: build/libs/*.jar

```

4. Artifact Management

4.1 Artifact Types

Type	Format	Storage
Java App	Jar / War	Nexus / Artifactory

Type	Format	Storage
Frontend	Static files / Docker image	Harbor / Docker Hub
Python	Wheel	Private PyPI
Docker Image	OCI image	Harbor
Helm Chart	Chart package	Harbor

4.2 Naming Convention

```
# Docker image
registry.company.com/{team}/{service-name}:{version}
# Example
registry.company.com/insurance/policy-service:v2.3.0
# Jar
com.company.insurance.{service}-{version}.jar
```

4.3 Versioning

Semantic Versioning 2.0:

```
{major}.{minor}.{patch}[-{pre-release}][+{build}]
Examples:
v2.3.0 # Release
v2.3.0-rc.1 # Release candidate
v2.3.0-alpha.1 # Alpha
v2.3.1 # Hotfix
```

Segment	Increment When
major	Breaking API changes, restructuring, major features
minor	Backward-compatible new features, non-critical interface changes
patch	Backward-compatible bug fixes

5. Environment Promotion

5.1 Environment Definitions

Environment	Purpose	Deploy Method	Data
dev	Developer self-test	Manual	Mock data
test	Functional test	Auto CI	Anonymized subset
staging	Pre-release verification	Auto CI	Anonymized (near production)
production	Production	Approval + auto deploy	Real data

5.2 Promotion Flow

```
dev > test > staging > production
Rules:
- Code must promote from main or release branch
- Each environment must pass all previous environment tests
- staging > production requires approval
- Hotfix may skip staging with approval
```

5.3 Approval Matrix

Promotion	Auto	Approver	Notes
dev > test	Yes (CI pass)	None	Auto-triggered
test > staging	Yes	Tech lead	Release tag trigger
staging > prod	No	Tech lead + QA + Architect	Release approval
hotfix > prod	Yes (emergency)	Emergency team	Post-approval required

6. Quality Gates

6.1 Gate Rules

Gate	Block Condition	Action
Build	Build failure	Block merge, notify author
Unit test	Failure / coverage below threshold	Block merge
Code scan	P0/P1 issues	Block merge
Integration	Core scenario failure	Block promotion
Security	High-risk vulnerability	Block promotion
Dependency	Known CVE	Block promotion (per severity)

6.2 Gate Exemptions

- P2+ issues may be exempted by tech lead
- High-risk security issues: must be fixed, no exemption
- Emergency fixes may temporarily bypass specific gates, must fix within 24h

7. Appendix

7.1 Reference Documents

- Google DevOps - Continuous Delivery
- GitLab CI/CD Documentation
- Semantic Versioning 2.0
- *Release & Change Management Standard*

7.2 Recommended Tools

Purpose	Recommended	Alternative
Repository	GitLab / GitHub	Bitbucket
CI Engine	GitHub Actions / GitLab CI	Jenkins
Artifact Repository	Nexus / Artifactory	Harbor
Container Registry	Harbor	Docker Registry

Purpose	Recommended	Alternative
Config Center	Nacos / Apollo	Consul
Deployment	Kubernetes + Helm	Docker Compose
IaC	Terraform	Pulumi / Ansible